

PC.v

The PC module takes the clock signal, reset signal, and the next PC value as inputs, and outputs current PC value. The PC is updated on the rising edge of the clock. If the reset signal is active, the PC is reset to 0. Otherwise, the PC is updated with the input pc_i during every clock cycle.

Adder.v

The Adder module performs a 32-bit addition. It takes two 32-bit inputs and outputs their sum. This module is used to compute $PC + 4$ for sequential instruction execution.

Registers.v

The Registers module reads data from two specified source registers (RS1 and RS2) and writes data to a destination register (RD) on the rising edge of the clock. If the reset signal is active (low), all register values are cleared.

Instruction_Memory.v

The Instruction_Memory module takes a 32-bit address input and outputs the 32-bit instruction located at that address. Instructions are word-aligned, so the address is right-shifted by 2 bits internally.

ALU_Control.v

The ALU_Control module takes a 10-bit funct field (from funct7 and funct3) and a 2-bit ALUOp as input, then determines the ALUOp number first to see if it is an I-type or R-type instruction. Then, it determines funct to see which function it matches to, and passes the 3-bit result to the ALU. Since there are only eight functions, we use a 3-bit control code to memorize that.

ALU.v

The ALU module takes two 32-bit operands and a 4-bit control signal as input. One operand comes from the register file, and the other is either a second register value or an immediate value selected via a multiplexer. The ALU performs the operation specified by the 3-bit control signal and outputs the result.

Control.v

The Control module takes the opcode as the input, then determines if it's an I-type or R-type instruction base on the opcode, and sets the three output: ALUOp, ALUSrc, and Reg_Write accordingly.

MUX32.v

The MUX32 module is a 2-to-1 multiplexer. It takes two 32-bit inputs and a single-bit selector. If the selector is 1, the output is the second input; if 0, the output is the first input. This is used to choose between register data and immediate values for ALU input.

Sign_Extend.v

The Sign_Extend module takes a 12-bit immediate as input and sign-extends it to 32 bits. It does so by copying the sign bit (the 11th bit) into the upper 20 bits. This allows the immediate to be correctly interpreted as a signed 32-bit value during ALU operations.

CPU.v

The CPU module is the top-level module that integrates all components: PC, Adder, Instruction_Memory, Registers, Sign_Extend, MUX32, ALU, Control, and ALU_Control. It defines the datapath and control logic needed to execute R-type and I-type instructions in a single cycle.